Technical Requirements Document

Automated Camera Trap Triage and Individual Tiger Movement Intelligence System

Pench Tiger Reserve

Document Version:	1.0
Date:	August 15, 2026
Classification:	Confidential - Technical
Document Type:	Technical Requirements Document (TRD)
Companion Document:	PRD v1.0 (Product Requirements Document)
Prepared For:	Engineering & Development Team
Domain:	Wildlife Conservation, Computer Vision, GIS, AI/ML

Table of Contents

1. Introduction & Scope

1.1 Purpose

This Technical Requirements Document (TRD) defines the detailed technical specifications, architecture, algorithms, data models, API contracts, infrastructure requirements, and engineering standards for the Automated Camera Trap Triage and Individual Tiger Movement Intelligence System. This document serves as the authoritative technical reference for the development, testing, and deployment teams.

1.2 Relationship to PRD

This TRD is the companion document to the Product Requirements Document (PRD v1.0). While the PRD defines WHAT the system must do (functional and non-functional requirements from a product perspective), this TRD defines HOW the system will achieve those requirements at a technical level. Every functional requirement (FR-x.x) in the PRD maps to one or more technical specifications in this document.

1.3 Scope Boundaries

>> **In Scope:** End-to-end image processing pipeline (ingestion through alerting), database design, API layer, dashboard backend, AI/ML model specifications, deployment architecture, security, and testing.

>> **Out of Scope:** Frontend UI/UX design specifications (covered in a separate UI Design Document), camera trap hardware procurement, field deployment logistics, and M-STrIPES integration protocol (pending NTCA approval).

1.4 Technical Standards & Conventions

> Programming Language: Python 3.11+ (backend, ML pipeline); TypeScript (frontend).

> API Standard: RESTful JSON API conforming to OpenAPI 3.1 specification.

> Date/Time: ISO 8601 format, stored in UTC, displayed in IST (UTC+5:30).

> Coordinate System: WGS 84 (EPSG:4326) for all geospatial data.

> Image Formats: JPEG (primary), PNG, TIFF, BMP (supported); WebP (internal thumbnails).

> Versioning: Semantic Versioning 2.0.0 (MAJOR.MINOR.PATCH).

> Logging: Structured JSON logging with OpenTelemetry trace context propagation.

> Documentation: All modules must include docstrings (Google style), type annotations, and README files.

2. System Architecture Overview

2.1 High-Level Architecture Diagram

The system is composed of six architectural layers organized in a pipeline topology. Data flows sequentially from ingestion through processing to analytics and presentation. Each layer is independently scalable and communicates via well-defined interfaces.

Architecture layers (top to bottom):

>> **Layer 1 - Presentation:** Web dashboard (React/Next.js), map interface (Leaflet/Mapbox GL JS), report generator, mobile-responsive alert viewer.

>> **Layer 2 - API Gateway:** FastAPI REST layer, WebSocket for real-time alerts, authentication middleware (JWT + RBAC), rate limiter (Redis-backed), OpenAPI documentation.

>> **Layer 3 - Application Services:** Orchestration service, processing run manager, human review workflow engine, alert dispatch service, export/report service.

>> **Layer 4 - AI/ML Inference:** MegaDetector V6 (blank filtering), Tiger Detector (fine-tuned YOLOv8), Flank Isolator (segmentation), Stripe Embedding Extractor (Siamese CNN), Matching Engine (vector similarity).

>> **Layer 5 - Data & Analytics:** PostgreSQL + PostGIS (relational), pgvector (embeddings), Redis (cache + task queue), MinIO (object storage), Spatial Analytics Engine (KDE, MCP, SECR).

>> **Layer 6 - Infrastructure:** Docker containers, NVIDIA Container Toolkit (GPU access), systemd services, backup daemon, monitoring stack (Prometheus + Grafana).

2.2 Component Interaction Model

Inter-component communication follows these patterns:

>> **Synchronous:** REST API calls for user-facing operations (dashboard queries, image retrieval, alert management). Timeout: 30 seconds.

>> **Asynchronous:** Celery task queue (Redis broker) for all AI/ML inference and batch processing. Tasks are submitted and tracked via task IDs. Workers consume from priority queues.

>> **Event-Driven:** PostgreSQL LISTEN/NOTIFY for real-time alert propagation to WebSocket connections. New alerts trigger immediate dashboard updates.

>> **File-Based:** MinIO object storage for image persistence. Processing pipeline reads/writes images via S3-compatible API. Thumbnails and crops are stored alongside originals.

2.3 Deployment Topology

Primary deployment: Single-server on-premise at PTR headquarters.

>> **Application Server:** Hosts FastAPI, Celery workers (CPU + GPU), React frontend (served via Nginx), and scheduler. Runs as Docker Compose stack.

>> **Database Server:** PostgreSQL 16 with PostGIS 3.4 and pgvector 0.5+. Can be co-located with application server for single-node deployment or separated for production scale.

>> **Object Storage:** MinIO instance (S3-compatible) for all image assets. Configured with erasure coding for data durability on a single node.

>> **Monitoring:** Prometheus metrics collector, Grafana dashboards, Loki for log aggregation. Lightweight stack suitable for single-server deployment.

>> **Backup:** Automated nightly database backups (pg_dump) and incremental MinIO backups to external USB/NAS storage. 30-day retention.

[DEPLOYMENT NOTE]

For reserves with reliable internet, the system can optionally sync processed results to a cloud replica (AWS/GCP) for disaster recovery and multi-reserve aggregation. All raw tiger location data remains on-premise.

3. Module 1: Blank Image Filtering Engine

3.1 Algorithm Specification

The blank filtering engine uses a two-stage approach combining object detection with statistical validation:

>> **Stage A - Object Detection:** Every input image is processed by MegaDetector V6 (or equivalent), which outputs bounding boxes with class labels (animal / person / vehicle) and confidence scores in range [0.0, 1.0].

>> **Stage B - Blank Classification:** An image is classified as BLANK if and only if: no bounding box from Stage A has a confidence score >= BLANK_THRESHOLD (configurable, default 0.2). Images with at least one detection >= threshold are classified as SUBJECT.

>> **Stage C - Boundary Review Flagging:** Images with max detection confidence in the range [BLANK_THRESHOLD - 0.05, BLANK_THRESHOLD + 0.05] are additionally flagged as BOUNDARY for optional human review.

>> **Stage D - Burst Sequence Logic:** If the camera's burst metadata is available (via EXIF or filename pattern), all frames in a burst sequence are retained if ANY single frame is classified as SUBJECT.

3.2 Model Architecture & Configuration

Primary model: MegaDetector V6 (MDv6)

>> **Base Architecture:** YOLOv9-C (compact variant) for on-premise deployment. RT-DETR variant available for cloud/high-GPU environments.

>> **Input Resolution:** 1280x1280 pixels (images resized with aspect-ratio-preserving padding).

>> **Output Classes:** 3 classes: 'animal' (class 1), 'person' (class 2), 'vehicle' (class 3).

>> **Inference Framework:** PyTorch 2.1+ via PytorchWildlife package. ONNX Runtime as fallback for CPU-only environments.

>> **Batch Size:** Dynamic batching: 8 (GPU, 16GB VRAM), 4 (GPU, 8GB VRAM), 1 (CPU fallback).

>> **Inference Speed Target:** < 50ms per image on NVIDIA RTX 4090; < 200ms on RTX 3060; < 2s on CPU (Intel i7).

>> **Fine-Tuning:** If domain shift exceeds 10% accuracy drop on Pench validation set, fine-tune MDv6 on minimum 3,000 Pench-specific labeled images using transfer learning (freeze backbone, train detection head for 50 epochs).

3.3 Quarantine Workflow

The quarantine system implements a safe, reversible deletion mechanism:

> QUARANTINE_ROOT directory mirrors the source directory structure.

> Blank images are MOVED (not copied) from source to quarantine, preserving relative paths.

> A quarantine manifest (JSON) records: original path, quarantine path, timestamp, model confidence, model version.

> Restore operation: any image in quarantine can be restored to its original location via API call or dashboard action.
> Purge policy: quarantined images older than QUARANTINE_RETENTION_DAYS (default: 90) can be permanently deleted via admin action with dual confirmation.
> Storage tracking: the system records cumulative bytes quarantined per run for reporting.

3.4 API Endpoints

Module 1 exposes the following REST endpoints:

```
POST   /api/v1/filtering/run
       Body: { "source_dir": "/path/to/images", "threshold": 0.2, "dry_run": false }
       Response: { "run_id": "uuid", "status": "queued" }

GET    /api/v1/filtering/runs/{run_id}
       Response: { "status": "completed", "total": 15000, "blanks": 11200,
                   "retained": 3800, "boundary": 230, "storage_saved_mb": 8400 }

GET    /api/v1/filtering/runs/{run_id}/report
       Response: Full JSON report with per-station breakdown and confidence histogram

POST   /api/v1/quarantine/{image_id}/restore
       Response: { "restored": true, "original_path": "/images/station_01/IMG_001.jpg" }

DELETE /api/v1/quarantine/purge
       Body: { "older_than_days": 90, "confirm": true }
       Response: { "purged_count": 5400, "freed_mb": 3200 }

GET    /api/v1/filtering/config
PUT    /api/v1/filtering/config
       Body: { "blank_threshold": 0.2, "boundary_margin": 0.05,
               "quarantine_retention_days": 90, "batch_size": 8 }
```

4. Module 2: Individual Tiger Identification Engine

4.1 Detection Pipeline

The tiger identification module uses a multi-stage detection and recognition pipeline:

>> **Stage 1 - Tiger Detection:** Fine-tuned YOLOv8-L model trained specifically on Indian tiger camera trap images. Input: full-frame retained image (post-blank-filtering). Output: bounding boxes for each tiger instance with confidence scores. Minimum detection confidence: 0.4.

>> **Stage 2 - Crop Extraction:** Each detected tiger bounding box is expanded by 15% padding on all sides (clamped to image boundaries) and cropped. The crop is resized to 448x448 pixels for downstream processing.

>> **Stage 3 - Quality Assessment:** Each crop is assessed for identification suitability: minimum resolution (200x200 pre-resize), blur detection (Laplacian variance > 100), occlusion estimation (< 30% body occluded). Low-quality crops are flagged but still processed with reduced confidence weighting.

>> **Stage 4 - Flank Detection & Isolation:** A secondary model (fine-tuned segmentation network or pose estimation) identifies the visible flank (left or right side). The flank region is isolated as the Region of Interest (ROI) for stripe extraction. Flank orientation (L/R) is recorded as metadata.

4.2 Stripe Pattern Feature Extraction

The stripe embedding extractor converts isolated flank images into fixed-dimensional feature vectors:

>> **Architecture:** Siamese Network with shared-weight CNN backbone. Primary backbone: DenseNet-169 pre-trained on ImageNet, fine-tuned on tiger re-identification dataset.

>> **Alternative Backbone:** InceptionResNetV2 (higher accuracy, slower inference) or EfficientNet-B4 (balanced performance). Selectable via configuration.

>> **Embedding Dimension:** 512-dimensional L2-normalized feature vector. Stored as float32 array in pgvector column.

>> **Training Loss:** Triplet Loss with online hard-negative mining. Margin parameter: 0.3. Training requires minimum 20 images per individual across 50+ individuals.

>> **Data Augmentation:** Random horizontal flip (only within same-flank sets), rotation (+/-15 degrees), brightness/contrast jitter (+/-20%), Gaussian blur, random crop (80-100% of flank area). Night-image-specific augmentation: gamma correction simulation.

>> **Input Preprocessing:** Flank crop resized to 224x224. Normalized to ImageNet mean/std. Histogram equalization applied for night images (detected via EXIF or brightness threshold < 50).

>> **Inference Speed:** < 100ms per crop on GPU; < 500ms on CPU. Batched inference supported (batch size 16 on 16GB VRAM).

4.3 Matching Algorithm & Catalogue Management

The matching engine compares new embeddings against the existing catalogue:

>> **Similarity Metric:** Cosine similarity between 512-d embedding vectors. Range: [-1.0, 1.0]. Higher = more similar.

>> **Vector Search:** pgvector extension with IVFFlat index (nlist=100, nprobe=10) for approximate nearest neighbor search. Falls back to exact search for catalogues < 500 individuals.

>> **Matching Thresholds:** AUTO_MATCH: cosine similarity >= 0.85 (applied automatically). REVIEW: 0.60 <= similarity < 0.85 (queued for human review). NEW_INDIVIDUAL: similarity < 0.60 against all catalogue entries (new enrollment triggered).

>> **Flank Matching Rule:** Only same-flank embeddings are compared (left-to-left, right-to-right). If both flanks are available for an individual, they are stored as separate embedding entries linked to the same individual ID.

>> **Multi-Image Consensus:** When multiple crops of the same individual exist in one capture event (burst mode), consensus matching is applied: the individual assignment is the mode of top-1 matches across all crops, weighted by confidence.

>> **Catalogue Update Policy:** On confirmed match (auto or human-verified): the individual's reference embedding is updated as a running weighted average (new embedding contributes 10% weight). This adapts to aging-related stripe changes.

>> **New Enrollment:** New individuals receive auto-generated IDs: PTR-T-{NNN} (e.g., PTR-T-001). The first capture becomes the primary reference embedding. Enrollment is marked 'provisional' until confirmed by a second independent capture.

4.4 API Endpoints

```
POST   /api/v1/identification/run
       Body: { "run_id": "filtering-run-uuid", "auto_match_threshold": 0.85 }
       Response: { "task_id": "uuid", "status": "queued" }

GET    /api/v1/identification/runs/{task_id}
       Response: { "status": "completed", "tigers_detected": 340,
                   "auto_matched": 280, "review_queued": 42, "new_enrolled": 18 }

GET    /api/v1/catalogue
       Query: ?page=1&per_page=20&status=active&sort=last_seen
       Response: Paginated list of individual profiles

GET    /api/v1/catalogue/{individual_id}
       Response: Full profile with capture history, embeddings metadata, images

GET    /api/v1/catalogue/{individual_id}/images
       Query: ?flank=L&station=ST-05&from=2025-01-01&to=2025-12-31

POST   /api/v1/review/queue
       Response: List of pending ambiguous matches with side-by-side data

PUT    /api/v1/review/{review_id}
       Body: { "action": "confirm|reject|merge", "target_id": "PTR-T-005" }

POST   /api/v1/catalogue/{individual_id}/merge
       Body: { "merge_with": "PTR-T-042", "reason": "duplicate" }
```

5. Module 3: Spatial Occupancy Analytics Engine

5.1 Home Range Computation

Home range estimation is performed for each identified individual using their complete capture history:

>> **Primary Method - KDE:** Fixed Kernel Density Estimation with bivariate normal kernel. Bandwidth selection: Least Squares Cross-Validation (LSCV) or plug-in estimator. Output: Utilization Distribution (UD) raster grid at 100m resolution.

>> **Isopleths:** 95% UD isopleth = total home range boundary. 50% UD isopleth = core activity area. Both exported as GeoJSON polygons with area in sq km.

>> **Secondary Method - MCP:** Minimum Convex Polygon computed from all capture points. Provided as a simpler alternative for quick visualization. Known limitation: overestimates range, especially with outlier captures.

>> **AKDE Option:** Autocorrelated KDE (using ctmm R package via rpy2 bridge) available for individuals with sufficient temporal capture data (>= 30 captures across >= 6 months). AKDE accounts for movement autocorrelation and provides confidence intervals on area estimates.

>> **Minimum Data Requirement:** KDE requires >= 10 unique capture locations. Individuals with fewer captures receive only point-map visualization (no range polygon). Warning flag raised for estimates based on 10-20 points.

>> **Centroid Computation:** Weighted centroid = sum(w_i * location_i) / sum(w_i), where w_i = recency weight (exponential decay, half-life = 90 days) * frequency weight at station. Output: single (lat, lon) point per individual.

>> **Implementation:** Python scipy.stats.gaussian_kde for standard KDE. R adehabitatHR::kernelUD via rpy2 for advanced options. Shapely for polygon operations. All computation triggered after each identification run completion.

5.2 Overlap Analysis

Territorial overlap between individuals is quantified and visualized:

>> **Pairwise Overlap Index:** Volume of Intersection (VI) between two individuals' UDs: VI(i,j) = integral of min(UD_i, UD_j). Range [0, 1]. Computed for all individual pairs with overlapping MCP bounding boxes (spatial pre-filter).

>> **Bhattacharyya's Affinity (BA):** Alternative overlap metric: BA = integral of sqrt(UD_i * UD_j). More sensitive to distribution shape differences. Both VI and BA stored for each pair.

>> **Overlap Matrix:** N x N matrix (N = number of active individuals) storing pairwise overlap values. Rendered as a color-coded heatmap in the dashboard.

>> **Spatial Overlap Polygons:** Intersection of 95% KDE isopleths between overlapping individuals rendered as shaded polygons on the map layer.

>> **Sex-Based Filtering:** If sex metadata is available, male-male, female-female, and male-female overlaps are categorized separately for ecological interpretation.

5.3 Map Rendering & Export

>> **Base Map:** Reserve boundary polygons (core + buffer) loaded from Shapefile/GeoJSON. Base tiles: OpenStreetMap (offline tile cache) or satellite imagery (Mapbox/Esri, if connectivity allows).

>> **Layer System:** Togglable layers: (1) Camera stations, (2) Individual KDE polygons (color-coded), (3) Core areas (50% isopleths), (4) Centroids, (5) Overlap zones, (6) Buffer/village boundaries, (7) Alert locations.

>> **Color Coding:** Each individual assigned a persistent color from a perceptually distinct palette (maximum 20 colors before recycling with pattern differentiation). Color assignments stored in database.

>> **Export Formats:** GeoJSON (all polygons + metadata), Shapefile (ESRI compatibility), KML (Google Earth), GeoTIFF (raster UD), PDF (static map with legend), CSV (tabular: individual, area, centroid, captures).

>> **Rendering Engine:** Backend: GeoPandas + Matplotlib for static PDF maps. Frontend: Leaflet.js with GeoJSON overlays for interactive web maps.

5.4 API Endpoints

```
POST   /api/v1/occupancy/compute
       Body: { "run_id": "identification-run-uuid", "kde_bandwidth": "lscv" }
       Response: { "task_id": "uuid", "status": "queued" }

GET    /api/v1/occupancy/individuals/{individual_id}
       Response: { "kde_95_area_sqkm": 45.2, "kde_50_area_sqkm": 12.8,
                   "mcp_area_sqkm": 62.1, "centroid": [21.78, 79.42],
                   "capture_count": 47, "kde_geojson": {...} }

GET    /api/v1/occupancy/overlap
       Query: ?individual_ids=PTR-T-001,PTR-T-005&metric=vi
       Response: { "overlap_matrix": [...], "overlap_polygons": {...} }

GET    /api/v1/occupancy/map/layers
       Response: GeoJSON FeatureCollection with all spatial layers

GET    /api/v1/occupancy/export
       Query: ?format=shapefile&individuals=all
       Response: Binary file download (ZIP for Shapefile)

GET    /api/v1/occupancy/summary
       Response: { "total_individuals": 62, "mean_range_sqkm": 38.5,
                   "median_range_sqkm": 35.2, "total_overlap_pairs": 18 }
```

6. Module 4: Deviation Detection & Alerting Engine

6.1 Baseline Computation

Each individual's historical baseline is computed from cumulative capture data:

>> **Established Home Range:** Rolling KDE 95% isopleth from all captures in the last 24 months (configurable window). Updated after each processing run.

>> **Station Visitation Profile:** For each individual: list of stations where captured, visit frequency per station, first/last visit timestamps. Stored as a JSON object in the individual's profile.

>> **Expected Capture Frequency:** Mean inter-capture interval (days) computed from historical data. Individuals with >= 5 captures have a computed 'regularity score' (coefficient of variation of inter-capture intervals).

>> **Seasonal Patterns:** If >= 2 years of data available: monthly capture probability estimated using generalized linear model. Flags seasonal vs. year-round residents.

>> **Baseline Freeze:** Baseline is NOT updated with data from the current run until alerts have been generated. This prevents circular baseline contamination.

6.2 Alert Rule Specifications

Four primary alert types are implemented:

Alert Type 1: Range Shift (RANGE_SHIFT)

> Trigger: Current run centroid distance from baseline centroid exceeds threshold.

> Core zone threshold: 15-20 sq km shift in KDE 95% area (configurable, default: 15 sq km).

> Buffer zone threshold: 5 km linear displacement of centroid (configurable).

> Computation: Haversine distance between current and baseline centroids. Area difference between current and baseline KDE 95% polygons.

> Output: Shift magnitude (km), direction (bearing), area change (sq km), visualization of old vs. new range.

Alert Type 2: Novel Station (NOVEL_STATION)

> Trigger: Individual captured at a station not in their established station visitation profile.

> Computation: Set difference between current capture stations and historical station set.

> Output: Novel station ID, distance from nearest historical station, distance from baseline range boundary.

> Confidence: HIGH if novel station is > 5 km from range boundary; MEDIUM if 2-5 km; LOW if < 2 km.

Alert Type 3: Buffer/Village Proximity (BUFFER_APPROACH)

> Trigger: Individual's current captures include stations classified as 'buffer-zone' or 'village-adjacent' that are NOT in their historical profile.

> Pre-requisite: Station metadata must include zone classification (core/buffer/village-adjacent).

> Priority: Always HIGH. Directly relevant to human-wildlife conflict early warning.

> Output: Station details, nearest village name and distance, individual's movement trajectory toward buffer.

Alert Type 4: Prolonged Absence (PROLONGED_ABSENCE)

> Trigger: Individual with regularity score >= 0.5 (captured in >= 3 of last 5 survey cycles) NOT detected in current run.
> Grace period: If current run is the first non-detection, alert is LOW confidence. Second consecutive non-detection: MEDIUM. Third+: HIGH.
> Output: Last known location, days since last capture, historical capture frequency, camera operational status at last-known stations.

6.3 Artefact Filtering Logic

Before generating alerts, the following survey artefacts are checked and suppressed:

> Camera Malfunction Check: If a station produced zero images (of any species) in the current run, any absence-based alerts referencing that station are suppressed. Alert metadata notes: 'Station offline'.
> Coverage Completeness: If the current run covers < 80% of historical stations, all PROLONGED_ABSENCE alerts are downgraded by one confidence level. Below 50% coverage, absence alerts are suppressed entirely.
> Known Relocations: If station metadata indicates a camera was relocated since the last run, novel-station alerts at the new location are suppressed if the old and new locations are within 500m.
> Seasonal Adjustment: If seasonal patterns are established and the current run falls in a historically low-capture season for an individual, absence alert confidence is reduced by one level.

6.4 API Endpoints

```
POST   /api/v1/alerts/generate
       Body: { "run_id": "occupancy-run-uuid" }
       Response: { "task_id": "uuid", "status": "queued" }

GET    /api/v1/alerts
       Query: ?type=RANGE_SHIFT&confidence=HIGH&status=new&page=1
       Response: Paginated alert list with full evidence

GET    /api/v1/alerts/{alert_id}
       Response: Full alert detail with evidence, supporting images, map data

PUT    /api/v1/alerts/{alert_id}
       Body: { "status": "acknowledged|resolved|false_positive", "notes": "..." }

GET    /api/v1/alerts/trends
       Query: ?individual_id=PTR-T-001&months=12
       Response: Time-series of alerts, range changes, capture frequency

POST   /api/v1/alerts/config
       Body: { "range_shift_core_sqkm": 15, "range_shift_buffer_km": 5,
               "absence_cycles": 3, "coverage_threshold": 0.8 }

GET    /api/v1/alerts/digest
       Query: ?format=pdf&period=weekly
       Response: PDF report of alerts in specified period
```

7. Database Architecture

7.1 Entity-Relationship Model

The database uses PostgreSQL 16 with PostGIS 3.4 and pgvector 0.5+ extensions. The schema is organized around six core entities with supporting tables for audit, configuration, and processing metadata.

Core entities and their relationships:

> individuals (1) --< (N) captures: One individual has many captures.

> stations (1) --< (N) captures: One station records many captures.

> individuals (1) --< (N) embeddings: One individual has multiple stored embeddings (per flank, per reference image).

> individuals (1) --< (N) alerts: One individual may trigger multiple alerts.

> processing_runs (1) --< (N) captures: One run processes many captures.

> processing_runs (1) --< (N) alerts: One run may generate multiple alerts.

> individuals (N) >--< (N) overlap_pairs: Many-to-many overlap relationships between individuals.

7.2 Schema Definitions

Primary table definitions (PostgreSQL DDL):

```
CREATE TABLE individuals (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tiger_id        VARCHAR(20) UNIQUE NOT NULL,  -- e.g., PTR-T-001
    assigned_name   VARCHAR(100),
    sex             VARCHAR(10),  -- male/female/unknown
    first_captured  TIMESTAMPTZ NOT NULL,
    last_captured   TIMESTAMPTZ NOT NULL,
    total_captures  INTEGER DEFAULT 0,
    status          VARCHAR(20) DEFAULT 'active',  -- active/absent/provisional
    baseline_range  GEOMETRY(POLYGON, 4326),  -- KDE 95% isopleth
    baseline_core   GEOMETRY(POLYGON, 4326),  -- KDE 50% isopleth
    centroid        GEOMETRY(POINT, 4326),
    range_area_sqkm FLOAT,
    metadata        JSONB,
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    updated_at      TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE captures (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    individual_id   UUID REFERENCES individuals(id),
    station_id      UUID REFERENCES stations(id) NOT NULL,
    image_path      TEXT NOT NULL,
    thumbnail_path  TEXT,
    crop_path       TEXT,
    location        GEOMETRY(POINT, 4326) NOT NULL,
    captured_at     TIMESTAMPTZ NOT NULL,
    flank_side      VARCHAR(5),  -- L/R/unknown
    match_confidence FLOAT,
```

```
    review_status   VARCHAR(20) DEFAULT 'auto',  -- auto/verified/pending
    reviewer_id     UUID,
    reviewed_at     TIMESTAMPTZ,
    quality_score   FLOAT,
    run_id          UUID REFERENCES processing_runs(id),
    metadata        JSONB,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE stations (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    station_code    VARCHAR(20) UNIQUE NOT NULL,
    station_name    VARCHAR(100),
    location        GEOMETRY(POINT, 4326) NOT NULL,
    zone            VARCHAR(20) NOT NULL,  -- core/buffer/village_adjacent
    nearest_village VARCHAR(100),
    village_dist_km FLOAT,
    deployment_status VARCHAR(20) DEFAULT 'active',
    last_maintenance TIMESTAMPTZ,
    metadata        JSONB
);
```

```
CREATE TABLE embeddings (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    individual_id   UUID REFERENCES individuals(id) NOT NULL,
    flank_side      VARCHAR(5) NOT NULL,  -- L/R
    vector          vector(512) NOT NULL,  -- pgvector type
    source_image    TEXT NOT NULL,
    is_reference    BOOLEAN DEFAULT false,
    confidence      FLOAT,
    model_version   VARCHAR(50),
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE alerts (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    alert_type      VARCHAR(30) NOT NULL,
    individual_id   UUID REFERENCES individuals(id),
    run_id          UUID REFERENCES processing_runs(id),
    confidence      VARCHAR(10) NOT NULL,  -- HIGH/MEDIUM/LOW
    status          VARCHAR(20) DEFAULT 'new',
    title           TEXT NOT NULL,
    description     TEXT,
    evidence        JSONB NOT NULL,  -- images, locations, measurements
    recommended_action TEXT,
    assigned_to     UUID,
    acknowledged_at TIMESTAMPTZ,
    resolved_at     TIMESTAMPTZ,
    notes           TEXT,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

7.3 Indexing Strategy

> B-tree indexes on all foreign keys and frequently queried columns (status, tiger_id, alert_type, captured_at).

> GiST spatial indexes on all GEOMETRY columns for fast spatial queries (ST_Within, ST_DWithin,

ST_Intersects).

> IVFFlat index on embeddings.vector column for approximate nearest neighbor search: CREATE INDEX ON embeddings USING ivfflat (vector vector_cosine_ops) WITH (lists = 100).
> Partial index on captures WHERE review_status = 'pending' for fast review queue retrieval.
> Composite index on (individual_id, captured_at DESC) for efficient capture history queries.
> BRIN index on captures.created_at for time-range partition pruning on large tables.

7.4 Vector Store Configuration

> Extension: pgvector 0.5+ compiled with PostgreSQL 16.
> Vector dimension: 512 (matching embedding extractor output).
> Distance metric: Cosine distance (1 - cosine_similarity) via <=> operator.
> Index type: IVFFlat for catalogues > 500 embeddings; exact scan for smaller catalogues.
> Index parameters: nlist = ceil(sqrt(N)) where N = total embeddings; nprobe = max(10, nlist/10).
> Query: SELECT individual_id, 1 - (vector <=> query_vector) AS similarity FROM embeddings WHERE flank_side = 'L' ORDER BY vector <=> query_vector LIMIT 10;

8. API Architecture & Specifications

8.1 REST API Design Principles

> All endpoints follow RESTful conventions with resource-oriented URLs.
> Request/response bodies use JSON (Content-Type: application/json). File uploads use multipart/form-data.
> HTTP status codes: 200 (success), 201 (created), 202 (accepted/queued), 400 (bad request), 401 (unauthorized), 403 (forbidden), 404 (not found), 422 (validation error), 429 (rate limited), 500 (server error).
> Pagination: Cursor-based for large collections. Parameters: ?cursor=xxx&per_page=20. Response includes next_cursor and total_count.
> Filtering: Query parameters for common filters (e.g., ?status=active&zone=core). Complex filters via POST with JSON body.
> Versioning: URL-based (e.g., /api/v1/...). Breaking changes increment major version.
> Documentation: Auto-generated OpenAPI 3.1 spec served at /api/docs (Swagger UI) and /api/redoc (ReDoc).

8.2 Authentication & Authorization

>> **Authentication:** JWT (JSON Web Token) bearer tokens issued by /api/v1/auth/login. Access tokens expire in 1 hour; refresh tokens in 7 days. Tokens contain user ID, role, and permissions claims.
>> **Password Storage:** bcrypt with work factor 12. Minimum password length: 12 characters.
>> **RBAC Roles:**
- ADMIN: Full system access including configuration, user management, and purge operations.
- BIOLOGIST: Read/write access to catalogue, occupancy, alerts. Can approve/reject review queue items.
- RANGE_OFFICER: Read access to all data. Write access to alerts (acknowledge/resolve). Export permissions.
- FIELD_STAFF: Upload images and view basic dashboard. No access to raw GPS coordinates or export functions.
- VIEWER: Read-only access to dashboards and reports. No data export or GPS coordinate visibility.

>> **API Key:** Service-to-service authentication uses API keys (256-bit random) for automated integrations (e.g., M-STrIPES sync). API keys are scoped to specific endpoints.

8.3 Rate Limiting & Throttling

> Rate limits enforced via Redis-backed sliding window counters.
> Default limits: 100 requests/minute for authenticated users, 20/minute for unauthenticated.
> Heavy endpoints (processing runs, exports): 5 requests/minute.
> Rate limit headers returned: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset.
> 429 responses include Retry-After header.

8.4 Master API Endpoint Catalogue

Complete endpoint listing organized by resource:

>> **Auth (3 endpoints):** /auth/login, /auth/refresh, /auth/logout

>> **Users (5 endpoints):** CRUD operations + role assignment for user management

>> **Filtering (6 endpoints):** Run management, quarantine operations, configuration (see Section 3.4)

>> **Identification (8 endpoints):** Run management, catalogue CRUD, review queue, merge (see Section 4.4)

>> **Occupancy (6 endpoints):** Compute, individual stats, overlap, map layers, export, summary (see Section 5.4)

>> **Alerts (7 endpoints):** Generate, list, detail, update, trends, config, digest (see Section 6.4)

>> **Stations (5 endpoints):** CRUD operations + bulk import for station metadata

>> **Runs (3 endpoints):** List all processing runs, run detail, run comparison

>> **Health (2 endpoints):** /health (liveness), /ready (readiness including DB and GPU checks)

>> **Total:** 45 endpoints across 9 resource groups

9. Data Ingestion & ETL Pipeline

9.1 File Ingestion Workflow

> Ingestion supports two modes: (A) Directory scan - operator points system at a mounted filesystem path; (B) Web upload - operator uploads ZIP archive via dashboard (max 10 GB per upload).
> Directory scan is recursive, processing all supported image files (JPEG, PNG, TIFF, BMP) in all subdirectories.
> Each ingestion creates a new processing_run record with status 'ingesting'.
> Files are copied (not moved) from source to MinIO object storage, preserving directory structure as object key prefixes.
> Original source files are never modified or deleted by the ingestion process.

9.2 EXIF & Metadata Extraction

> EXIF data extracted using Python Pillow (PIL) and exifread libraries.
> Extracted fields: DateTimeOriginal (capture timestamp), Make/Model (camera), ImageWidth/Height, GPS coordinates (if embedded), Orientation.
> If EXIF timestamp is missing, file modification time is used as fallback (with 'estimated' flag).
> Station assignment: (A) If GPS is in EXIF, nearest station within 100m radius is assigned. (B) If no GPS, station is inferred from directory name pattern (configurable regex). (C) Manual override via CSV upload mapping filenames to stations.
> Camera serial number (if in EXIF) is recorded for camera health tracking.

9.3 Deduplication Strategy

> Perceptual hashing (pHash) computed for every ingested image (64-bit hash).
> Exact duplicates (Hamming distance = 0) are detected and skipped, with the duplicate count recorded in the run report.
> Near-duplicates (Hamming distance <= 5) are flagged but ingested, as burst-mode frames may be perceptually similar but contain useful pose variation.
> Cross-run deduplication: new images are checked against the last 3 runs to prevent re-processing of previously ingested images.

9.4 Data Validation Rules

> Image must be decodable (corrupt files logged and skipped).
> Minimum resolution: 640x480 pixels. Smaller images logged as 'low_resolution' and processed with quality flag.
> Maximum file size: 50 MB per image (larger files logged and skipped - likely non-camera-trap).
> Timestamp sanity: capture time must be within [2010-01-01, current_date + 1 day]. Out-of-range timestamps flagged.
> GPS sanity: if GPS is present, coordinates must fall within Pench bounding box (+/- 0.5 degrees tolerance).

Out-of-bounds GPS logged as 'gps_anomaly'.

10. Infrastructure & Deployment

10.1 Hardware Specifications

Minimum and recommended hardware for the on-premise processing server:

Minimum Configuration (up to 100K images/run)

- CPU: Intel Core i7-12700 or AMD Ryzen 7 5800X (8+ cores)
- RAM: 32 GB DDR4-3200
- GPU: NVIDIA RTX 3060 12GB VRAM
- Storage: 2 TB NVMe SSD (OS + application) + 4 TB HDD (image archive)
- Network: Gigabit Ethernet
- UPS: 1500VA minimum for graceful shutdown

Recommended Configuration (up to 500K images/run)

- CPU: Intel Xeon W-2245 or AMD Ryzen 9 7950X (16+ cores)
- RAM: 64 GB DDR5-4800
- GPU: NVIDIA RTX 4090 24GB VRAM or NVIDIA A4000 16GB
- Storage: 2 TB NVMe SSD (OS + DB) + 8 TB NVMe/SATA SSD (MinIO) + 16 TB HDD (backups)
- Network: Gigabit Ethernet + 4G/LTE modem for remote alerting
- UPS: 3000VA with network management card

10.2 Software Dependencies

Core software stack with version requirements:

- OS: Ubuntu 22.04 LTS Server (recommended) or Windows Server 2022 with WSL2.
- Container Runtime: Docker 24+ with Docker Compose v2. NVIDIA Container Toolkit 1.14+ for GPU access.
- Python: 3.11+ (3.12 recommended). Virtual environment via venv or conda.
- PyTorch: 2.1+ with CUDA 12.1 support. Torchvision 0.16+.
- PostgreSQL: 16+ with PostGIS 3.4+, pgvector 0.5+.
- Redis: 7.2+ (task broker and cache).
- MinIO: Latest stable (S3-compatible object storage).
- Nginx: 1.24+ (reverse proxy and static file serving).
- Node.js: 20 LTS (frontend build toolchain).

10.3 Containerization Strategy

The application is deployed as a Docker Compose stack with the following services:

```
services:
  api:          FastAPI application server (2 workers, Uvicorn)
  worker-gpu:   Celery worker with GPU access (1 instance, concurrency=1)
  worker-cpu:   Celery worker for non-GPU tasks (2 instances, concurrency=4)
  scheduler:    Celery Beat for scheduled tasks (backup, cleanup)
```

```
postgres:     PostgreSQL 16 + PostGIS + pgvector
redis:        Redis 7.2 (broker + cache)
minio:        MinIO object storage (single node)
nginx:        Reverse proxy + frontend static files
prometheus:   Metrics collection
grafana:      Monitoring dashboards
```

> GPU worker container uses nvidia/cuda:12.1-runtime-ubuntu22.04 base image with NVIDIA Container Toolkit runtime.
> Persistent volumes: postgres_data, minio_data, redis_data, model_weights, quarantine_storage.
> Resource limits: GPU worker pinned to GPU 0; CPU workers limited to 4 cores each; PostgreSQL allocated 16 GB shared_buffers.

10.4 Network Architecture

> All services communicate on an internal Docker bridge network (172.18.0.0/16). No service ports exposed to WAN except Nginx (443/HTTPS, 80/HTTP redirect).
> Nginx terminates TLS (Let's Encrypt or self-signed certificate for air-gapped deployment).
> PostgreSQL listens only on internal network. External access (e.g., for R/QGIS clients on same LAN) via SSH tunnel or VPN.
> Alert SMS delivery via cellular modem (GSM AT commands) or SMS gateway API (Twilio/MSG91) over 4G uplink.
> Optional: WireGuard VPN for secure remote access by administrators.

11. Security Architecture

11.1 Encryption Standards

> Data at rest: AES-256-GCM encryption for PostgreSQL (Transparent Data Encryption) and MinIO (server-side encryption with KMS). Encryption keys stored in a dedicated key file with 0600 permissions.
> Data in transit: TLS 1.3 for all HTTP traffic (Nginx termination). TLS 1.2+ for internal PostgreSQL connections (sslmode=verify-full).
> Backup encryption: All database dumps and image backups encrypted with GPG (RSA-4096) before writing to external storage.
> Sensitive fields: GPS coordinates in API responses are rounded to 3 decimal places (~111m precision) for RANGE_OFFICER and below roles. Full precision available only to ADMIN and BIOLOGIST roles.

11.2 Access Control (RBAC)

Role-based access control matrix (key resources):

> ADMIN: Full CRUD on all resources. User management. System configuration. Purge operations. Raw GPS export.
> BIOLOGIST: Read/write catalogue, captures, occupancy. Approve reviews. Generate reports. Raw GPS access.
> RANGE_OFFICER: Read all. Write alerts (acknowledge/resolve). Export reports (GPS rounded). Upload images.
> FIELD_STAFF: Upload images. View dashboard (no GPS). View assigned alerts.
> VIEWER: Read-only dashboard and reports. No GPS. No export.
> All role assignments and changes are logged in the audit trail with actor ID and timestamp.

11.3 Audit Logging

> Every API request logged with: timestamp, user ID, role, endpoint, method, request body hash (SHA-256), response status, IP address, user agent.
> Data modification events logged with: before/after values for critical fields (individual status, match decisions, alert status).
> Audit logs stored in a dedicated PostgreSQL table (audit_log) with append-only policy. No DELETE or UPDATE operations permitted on audit_log.
> Log retention: 2 years minimum. Archived to compressed files after 6 months.
> Failed authentication attempts logged and rate-limited (5 failures in 15 minutes triggers 30-minute account lockout).

11.4 Vulnerability Management

> Container images scanned with Trivy on every build. Critical/High CVEs must be resolved before deployment.
> Python dependencies audited with pip-audit weekly. Alerts for known vulnerabilities.

> PostgreSQL and Redis updated to latest patch versions within 30 days of release.

> No default credentials. All service passwords generated as 32+ character random strings during initial deployment.

> SSH access to server via key-based authentication only. Password authentication disabled. Root login disabled.

12. Performance Engineering

12.1 Benchmarks & SLAs

Performance targets for each system component:

>> **Blank Filtering:** Throughput: >= 1,000 images/minute (GPU), >= 100 images/minute (CPU). Latency p95: < 60ms/image (GPU).

>> **Tiger Detection:** Throughput: >= 500 images/minute (GPU). Latency p95: < 120ms/image.

>> **Embedding Extraction:** Throughput: >= 200 crops/minute (GPU). Latency p95: < 300ms/crop.

>> **Vector Search:** Latency p95: < 50ms for top-10 nearest neighbor query against 10,000 embeddings.

>> **KDE Computation:** < 30 seconds per individual with 100 capture points. < 30 minutes for full reserve (80 individuals).

>> **API Response:** p95 latency < 200ms for read endpoints. < 500ms for write endpoints. < 2s for complex spatial queries.

>> **Dashboard Load:** Initial page load < 3 seconds. Map render with all layers < 5 seconds.

12.2 GPU Optimization

> Mixed-precision inference (FP16) enabled for all detection and embedding models. Reduces VRAM usage by ~40% with < 1% accuracy impact.

> Dynamic batching: images are accumulated in a queue and dispatched to GPU in optimal batch sizes (determined by available VRAM at runtime).

> Model warmup: on service startup, each model processes 5 dummy images to initialize CUDA kernels and memory allocators.

> ONNX Runtime with TensorRT execution provider as optional high-performance inference backend for NVIDIA GPUs.

> GPU memory monitoring: if VRAM usage exceeds 90%, batch size is automatically reduced. OOM errors trigger graceful fallback to smaller batch size.

12.3 Caching Strategy

> Redis cache for: API response caching (TTL: 5 minutes for dashboard data, 1 hour for catalogue), session tokens, rate limit counters.

> Thumbnail cache: 256x256 WebP thumbnails generated on first access and cached in MinIO. Subsequent requests served directly.

> Spatial query cache: KDE polygons (GeoJSON) cached per individual with invalidation on new data ingestion.

> Embedding cache: frequently queried reference embeddings loaded into Redis (serialized numpy arrays) for sub-millisecond retrieval.

12.4 Batch Processing Optimization

> Pipeline parallelism: Stage 1 (blank filtering) and Stage 2 (tiger detection) run on separate Celery queues.

Stage 1 output feeds Stage 2 via a shared Redis queue.

> I/O optimization: images read from MinIO in parallel (thread pool, 8 concurrent downloads). Decoded using Pillow with JPEG turbo decoder.

> Database bulk operations: captures inserted in batches of 500 (executemany). Spatial index updates deferred until batch completion.

> Progress tracking: processing status updated every 100 images (not per image) to minimize database write overhead during large runs.

> Resumable processing: if a run is interrupted, it can be resumed from the last checkpoint (tracked by last processed image ID).

13. Testing Strategy

13.1 Unit Testing

- Framework: pytest 7+ with pytest-cov for coverage measurement.
- Coverage target: >= 85% line coverage for all non-ML application code.
- Test categories: data validation, API serialization, database queries, spatial computations, alert logic, quarantine operations.
- Mocking: pytest-mock for external service dependencies (MinIO, Redis). Factory Boy for database fixture generation.
- Execution: all unit tests must pass in < 5 minutes on CI without GPU.

13.2 Integration Testing

- Framework: pytest with Docker Compose test environment (PostgreSQL, Redis, MinIO containers).
- Test scenarios: complete ingestion-to-alert pipeline with 100 test images, API endpoint integration with authentication, WebSocket alert delivery, export file generation and validation.
- Database: test database with seeded data (10 individuals, 500 captures, 5 stations). Rolled back after each test suite.
- Execution: integration tests run in < 15 minutes. GPU-dependent tests skipped on CPU-only CI runners (with @pytest.mark.gpu decorator).

13.3 Model Validation Testing

- Holdout test set: 20% of labeled Pench data reserved for validation (stratified by species, time-of-day, season).
- Blank classifier metrics: precision, recall, F1-score, confusion matrix. Tested at multiple threshold values (0.1, 0.15, 0.2, 0.25, 0.3).
- Tiger detector metrics: mAP@0.5, mAP@0.5:0.95 on Pench test set. Breakdown by image quality (day/night, distance, occlusion).
- Re-identification metrics: Rank-1, Rank-5, Rank-10 accuracy, mAP. Evaluated in closed-set (all individuals seen in training) and open-set (novel individuals) scenarios.
- Regression testing: after any model update, all validation metrics must meet or exceed previous version. Automated comparison report generated.
- Bias testing: accuracy disaggregated by: day vs. night images, adult vs. cub, left vs. right flank, image quality quartiles.

13.4 Load & Stress Testing

- Tool: Locust or k6 for API load testing.
- Scenarios: (A) 10 concurrent users browsing dashboard for 30 minutes. (B) Bulk ingestion of 50,000 images while 5 users access dashboard. (C) 100 concurrent API requests to occupancy/map endpoint.
- Success criteria: no HTTP 5xx errors, p95 latency within SLA, memory usage below 80% of available RAM,

GPU utilization < 95% sustained.

> Endurance test: 24-hour continuous processing of simulated survey data (recycled test images) to detect memory leaks or resource exhaustion.

14. Monitoring & Observability

14.1 Metrics (Prometheus)

> Application metrics: request count/latency (per endpoint), active Celery tasks, queue depth, processing run progress, images processed/second.
> ML metrics: inference latency (per model), batch size, GPU utilization, VRAM usage, model version in use, blank ratio per run.
> Infrastructure metrics: CPU/memory/disk usage, PostgreSQL connections/query time, Redis memory/hit rate, MinIO storage usage.
> Business metrics: total individuals in catalogue, new enrollments this month, alert count by type/confidence, human review backlog.
> Scrape interval: 15 seconds for application metrics, 60 seconds for infrastructure metrics.

14.2 Logging (Loki)

> All application logs in structured JSON format with fields: timestamp, level, service, trace_id, message, metadata.
> Log levels: DEBUG (development only), INFO (standard operations), WARNING (non-critical issues), ERROR (failures requiring attention), CRITICAL (system-down scenarios).
> Log retention: 90 days in Loki, 1 year in compressed archive on external storage.

14.3 Alerting (Grafana)

> System alerts (distinct from tiger behavioural alerts): GPU OOM, disk usage > 85%, PostgreSQL connection exhaustion, Celery worker crash, API error rate > 5%.
> Notification channels: email (SMTP), webhook (for integration with existing forest department systems).
> Dashboard panels: real-time processing progress, model accuracy trends, system health overview, alert volume over time.

15. Disaster Recovery & Backup

15.1 Backup Strategy

>> **Database:** Automated pg_dump every 24 hours (2 AM IST). Full dump + WAL archiving for point-in-time recovery. Backup compressed with zstd and encrypted with GPG.

>> **Object Storage (Images):** MinIO replication to external NAS/USB drive. Incremental sync using mc mirror (MinIO Client). Run weekly or after each major processing run.

>> **Configuration:** Docker Compose files, environment variables, model weights, and SSL certificates backed up to version-controlled repository (Git). Updated on every configuration change.

>> **Retention:** Daily database backups: 30 days. Weekly image backups: 90 days. Monthly full system backup: 1 year.

15.2 Recovery Procedures

>> **RPO (Recovery Point Objective):** Database: 24 hours (last backup). Images: 7 days (last sync). Configuration: 0 hours (version controlled).

>> **RTO (Recovery Time Objective):** Full system restoration from backup: < 4 hours on same hardware. < 8 hours on replacement hardware (including OS and Docker setup).

>> **Procedure:** (1) Restore Docker Compose configuration from Git. (2) Restore PostgreSQL from latest dump. (3) Restore MinIO data from external storage. (4) Verify data integrity with checksums. (5) Run health checks on all services. (6) Resume processing from last checkpoint.

15.3 Data Integrity

> All database backups verified with pg_restore --list (structure check) and row-count comparison against live database.

> MinIO objects verified with MD5 checksums on backup media vs. live storage (sampled 1% per run).

> Monthly disaster recovery drill: restore from backup on isolated VM and run integration test suite.

16. CI/CD Pipeline

16.1 Pipeline Stages

>> **Stage 1 - Lint & Format:** Ruff (linting + formatting), mypy (type checking), eslint + prettier (frontend). Must pass with zero warnings.

>> **Stage 2 - Unit Tests:** pytest with coverage. Minimum 85% coverage. Runs on CPU-only runner. < 5 minutes.

>> **Stage 3 - Integration Tests:** Docker Compose test stack. API tests, database migration tests, pipeline integration tests. < 15 minutes.

>> **Stage 4 - Security Scan:** Trivy for container images. pip-audit for Python dependencies. npm audit for frontend. Block on Critical/High findings.

>> **Stage 5 - Build & Push:** Docker images built with multi-stage Dockerfiles. Tagged with Git commit SHA and semantic version. Pushed to private registry (Harbor or Docker Hub private repo).

>> **Stage 6 - Deploy:** Manual trigger for production deployment. Automated for staging/dev. Blue-green deployment with health check verification before traffic switch.

16.2 Branch Strategy

> main: production-ready code. Protected branch, requires PR with 1+ approvals.

> develop: integration branch. Automated deployment to staging environment.

> feature/*: individual feature branches. PR to develop when ready.

> hotfix/*: emergency fixes branched from main. PR to both main and develop.

> Database migrations: managed by Alembic. Migration files committed alongside application code. Auto-applied on deployment.

17. Edge Computing Architecture (Future Phase)

This section outlines the technical architecture for optional edge-computing deployment at high-priority camera stations. This is a future-phase capability, not required for initial deployment.

17.1 Edge Device Specification

>> **Primary Device:** NVIDIA Jetson Orin Nano (8GB). 40 TOPS AI performance. 15W power envelope. -25C to +80C operating range.

>> **Alternative:** Google Coral Dev Board (Edge TPU). Lower power (2W inference) but limited to TFLite models.

>> **Storage:** 256 GB microSD or NVMe SSD for local image buffer.

>> **Connectivity:** LoRaWAN for metadata/alerts (low bandwidth, long range). 4G/LTE cellular for image transmission (where coverage exists).

>> **Power:** Solar panel (50W) + LiFePO4 battery (100Wh). Supports 24/7 operation with 2 days autonomy.

>> **Enclosure:** IP67 weatherproof housing. Anti-tamper screws. GPS for device tracking.

17.2 Edge Processing Pipeline

> On-device blank filtering using MegaDetector V6 compact (YOLOv9-T, optimized for Jetson via TensorRT).

> Only non-blank images transmitted to central server, reducing bandwidth by 70-95%.

> Metadata alerts (animal detected, human detected) sent immediately via LoRaWAN (< 100 bytes per alert).

> Images queued for transmission during high-bandwidth windows (scheduled cellular upload).

> Local buffer retains all images for 30 days, ensuring no data loss even if transmission fails.

17.3 Edge-Cloud Synchronization

> Edge devices register with central server via REST API on first connection.

> Model updates pushed to edge devices via OTA (over-the-air) mechanism when connected.

> Device health (battery, storage, temperature, model version) reported hourly via LoRaWAN.

> Central server maintains a device registry with last-seen timestamps. Offline > 48 hours triggers maintenance alert.

18. Technical Constraints & Limitations

18.1 Known Limitations

> Night image quality: infrared camera trap images have lower contrast and no color information, reducing both detection and re-identification accuracy by an estimated 10-20%.

> Single-flank matching: the system can only match flanks of the same side (L-to-L, R-to-R). A tiger photographed only from the left cannot be matched to a catalogue entry with only right-flank images. Cross-flank matching is a research frontier not yet production-ready.

> Cub identification: stripe patterns in cubs may change subtly during growth (first 12-18 months).

Re-identification accuracy for cubs is expected to be lower. Cubs should be tracked with 'provisional' status until patterns stabilize.

> Occlusion and pose: heavily occluded tigers (>50% body hidden by vegetation) or tigers in unusual poses (swimming, climbing) may fail detection or produce low-quality crops unsuitable for identification.

> Camera trap density: home range estimates are only as accurate as the spatial distribution of camera stations. Gaps in camera coverage create blind spots in occupancy maps.

18.2 Technical Constraints

> Single GPU deployment: the system is designed for single-GPU inference. Multi-GPU scaling requires modification to the Celery worker configuration and model serving layer.

> PostgreSQL scaling: single-node PostgreSQL is adequate for the expected data volume (< 1M captures). Horizontal scaling (Citus or read replicas) needed only for multi-reserve aggregation.

> Real-time processing: the system operates in batch mode (process after data collection). Near-real-time processing (< 1 hour latency) requires edge computing infrastructure (Phase 2).

> Internet dependency: alert delivery (email, SMS) requires internet connectivity. On-premise dashboard functions fully offline, but remote access requires VPN or internet.

18.3 Scalability Boundaries

> Tested for: up to 500,000 images per processing run, 200 individual tiger profiles, 10,000+ embeddings, 500 camera stations.

> Beyond these thresholds: database query optimization, embedding index tuning, and potentially horizontal scaling will be required.

> Multi-reserve deployment: each reserve should have its own database instance. Cross-reserve queries via federated query layer (future phase).

19. Appendices

19.1 Appendix A: Environment Variables Reference

```
# Database
DATABASE_URL=postgresql://user:pass@postgres:5432/tigersys
POSTGRES_MAX_CONNECTIONS=100

# Redis
REDIS_URL=redis://redis:6379/0
CELERY_BROKER_URL=redis://redis:6379/1

# MinIO
MINIO_ENDPOINT=minio:9000
MINIO_ACCESS_KEY=<generated>
MINIO_SECRET_KEY=<generated>
MINIO_BUCKET=camera-trap-images

# AI/ML
MEGADETECTOR_MODEL_PATH=/models/mdv6_yolov9c.pt
TIGER_DETECTOR_MODEL_PATH=/models/tiger_yolov8l.pt
EMBEDDING_MODEL_PATH=/models/siamese_densenet169.pt
INFERENCE_DEVICE=cuda:0  # or cpu
BLANK_THRESHOLD=0.2
AUTO_MATCH_THRESHOLD=0.85
REVIEW_THRESHOLD=0.60

# Security
JWT_SECRET_KEY=<generated-256-bit>
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=60
JWT_REFRESH_TOKEN_EXPIRE_DAYS=7

# Alerting
SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMS_GATEWAY_URL=https://api.msg91.com/v5/flow
SMS_API_KEY=<key>
ALERT_EMAIL_RECIPIENTS=fd@pench.gov.in,biologist@pench.gov.in
```

19.2 Appendix B: Docker Compose Service Ports

- Nginx: 80 (HTTP), 443 (HTTPS) - exposed to host network
- FastAPI: 8000 (internal only)
- PostgreSQL: 5432 (internal only, optional host exposure for admin tools)
- Redis: 6379 (internal only)
- MinIO: 9000 (API, internal), 9001 (Console, internal)
- Prometheus: 9090 (internal only)
- Grafana: 3000 (internal, proxied via Nginx at /monitoring)

19.3 Appendix C: Database Migration Strategy

> All schema changes managed by Alembic (SQLAlchemy migration tool).
> Migration naming: {timestamp}_{description}.py (e.g., 20260801_add_baseline_range_column.py).
> Every migration must include both upgrade() and downgrade() functions for reversibility.
> Migrations tested in CI against a fresh database (empty) and against a seeded database (with test data).
> Production migrations executed automatically on deployment via alembic upgrade head in the Docker entrypoint.
> Data migrations (backfills) separated from schema migrations. Data migrations are idempotent.

19.4 Appendix D: Model Versioning & Registry

> Model weights stored in MinIO bucket: /models/{model_name}/{version}/weights.pt
> Model metadata (architecture, training dataset, hyperparameters, validation metrics) stored in a models table in PostgreSQL.
> Active model version recorded in system configuration. Hot-swap enabled: changing active version takes effect on next inference request without restart.
> Model rollback: previous version always retained. One-click rollback via admin API.
> Model performance tracking: per-run accuracy metrics logged against model version for degradation detection.

--- End of Document ---

Technical Requirements Document v1.0

Automated Camera Trap Triage & Tiger Movement Intelligence System

Pench Tiger Reserve
